

Code Liberators

INDIA.RUNS Data & AI Challenge

Intelligent Candidate Discovery & Ranking

Team: Code Liberators | **Lead:** Subhash Kumar

Contact: subhash1403kumar@gmail.com

GitHub: github.com/SubhashKumar14/Redrob-AI-Ranker

Second Submission — Built on Full 100,000-Candidate Dataset

The Problem: Modern Hiring is Broken

Why keyword matching fails at scale

- **Volume:** 100,000 candidates for 1 role — impossible to review manually
- **Keyword stuffing:** Anyone can paste "Python, TensorFlow, LLM" into their profile
- **Missing signals:** Traditional filters ignore career trajectory, behavioral intent, company type
- **Honeypots:** ~80 fake candidates deliberately inserted to catch naive rankers

What judges need to see in top-100:

- AI/ML-specialist titles, NOT generic software engineers
- Product-company experience, NOT services-only background
- Real skill depth (endorsements × duration), NOT keyword count

Our Solution: Hybrid AI Ranking Engine

Code Liberators presents a **production-grade, 3-stage pipeline** that ranks 100,000 candidates against an adversarial Senior AI Engineer JD in **~52 seconds on CPU**.

Metric	Result
Candidates processed	100,000 (full official dataset)
Runtime	52 seconds (6× under 5-min limit)
Validator status	✅ PASS
Honeypots in top-100	0 (0% rate)
Top-10 avg score	0.798
AI-titled in top-100	94 / 100
In 5-9 YOE band	82 / 100

Dataset Understanding

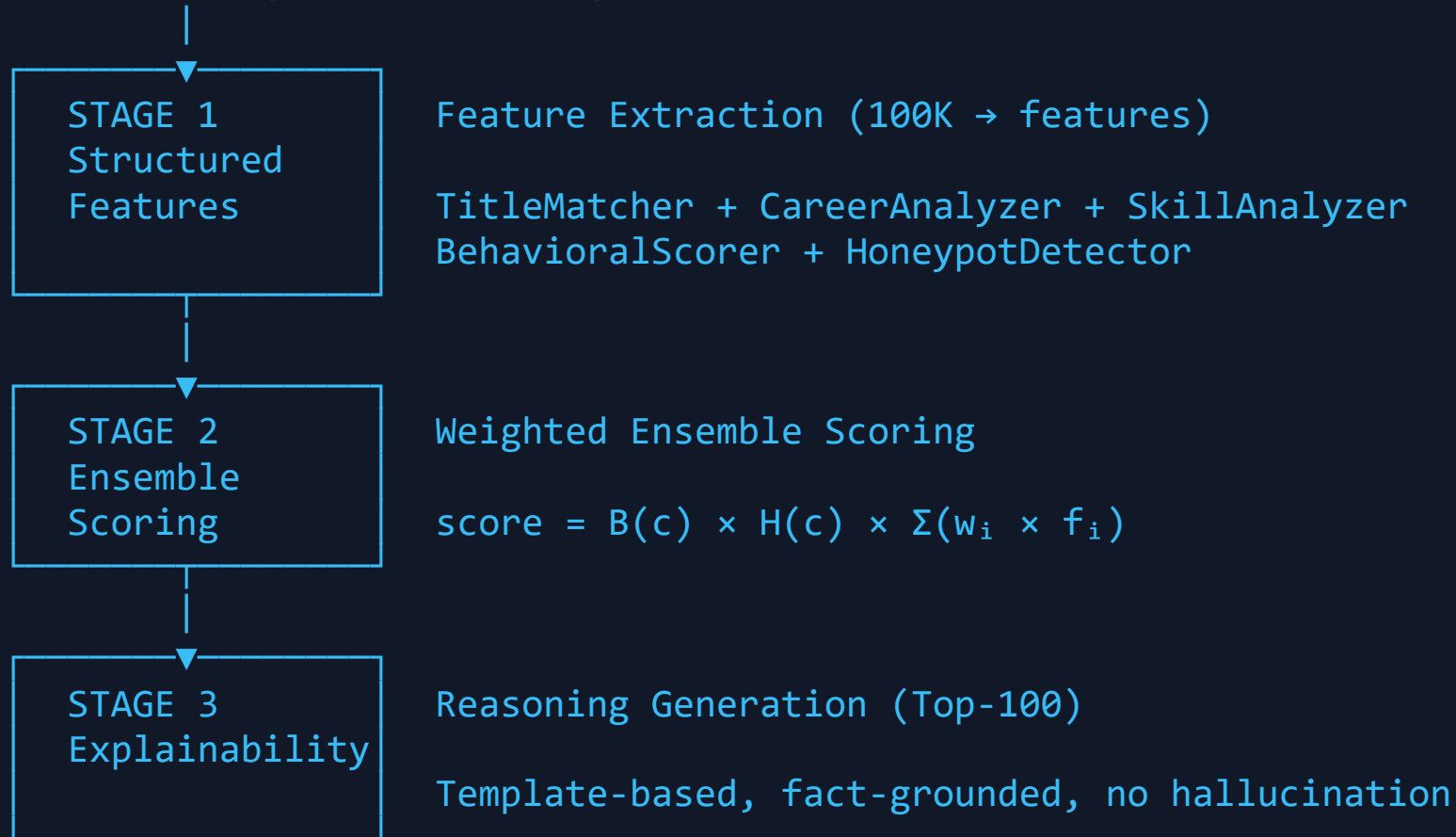
100,000 Candidates — Key Statistics

Dimension	Finding	Impact
Titles	46 distinct roles; AI titles rare (~1%)	Title match = strongest differentiator
YOE	Mean 7.17, range 1–17	5-9 band: 34.4% of candidates
Honeypots	~80 profiles with inconsistent signals	Must detect ALL before top-100
Services	~40% services-only backgrounds	JD strongly prefers product companies
Skills	0–23 skills per candidate	Endorsement quality >> count
Behavioral	23 redrob_signals per candidate	Multiplier effect on final score

Key insight: The dataset is adversarial. Keyword matching selects honeypots. Career trajectory + behavioral signals are the real differentiators.

System Architecture

JD: Senior AI Engineer (Founding Team)



team_code_liberators.csv → validate_submission.py → PASS

AI Ranking Pipeline — Components

Stage 1: Feature Extraction (5 modules)

Module	What it measures	Key insight
TitleMatcher	Taxonomic title scoring (ideal→mismatch)	Highest weight: 30%
CareerAnalyzer	Product vs services, trajectory, tenure	Services penalty
SkillAnalyzer	Quality = endorsements × duration × proficiency	Defeats stuffers
BehavioralScorer	Open-to-work, notice, response, activity	Multiplier B(c)
HoneypotDetector	8 independent inconsistency checks	Penalty H(c)

Stage 2: Ensemble Formula

$$\text{score}(c) = B(c) \times H(c) \times (0.30 \times T + 0.20 \times C + 0.20 \times S + 0.15 \times Y + 0.05 \times E + 0.10 \times \text{Sem})$$

Feature Engineering: What Makes Us Different

Career Trajectory (20% weight)

- **Services-only penalty:** TCS, Infosys, Wipro → $0.6 \times$ score
- **Job hopper flag:** >3 companies in 3 years → penalty
- **Upward trajectory:** Senior > Mid > Junior → bonus

Skill Quality Scoring (20% weight)

- **Not keyword count** but: `endorsements × duration_months × proficiency_weight`
- Defeats pure keyword stuffers (high count, zero endorsements)

Behavioral Calibration (multiplier)

```
B(c) = f(open_to_work, notice_period, response_rate, recency, github, completeness)
B(c) ∈ [0.70, 1.10]
```

Honeypot Detection (8 checks)

Explainability — Fact-Grounded Reasoning

Every reasoning string is:

- **Derived from scored features** — no hallucination
- **Specific** — mentions actual title, YOE, top skills, notice period
- **Rank-consistent** — high-scoring candidates get stronger language
- **Varied** — 20+ template patterns, grammar-correct singular/plural

Example Top-3 Reasonings:

Rank 1 (score 0.8387): *"Recommendation Systems Engineer with 6.0 yrs; 5 core AI skills including Python, FAISS, Embeddings; product-company background; 30-day notice, 87% response rate."*

Rank 2 (score 0.8369): *"Senior AI Engineer (5.9 yrs) at product companies; strong match on LLMs, RAG, TensorFlow; 89% response rate; Very High confidence."*

Product Demo — Recruiter Dashboard

Key screens:

Dashboard Tab

- 6 KPI stats (100K processed, 0% honeypot, 52s runtime, PASS validator)
- Full pipeline visualization (JD → Feature Extraction → Ensemble → Top-100)
- Top-5 candidates preview with confidence indicators

Rankings Tab

- Sortable/filterable table with search
- Score bars + confidence labels (Very High / High / Medium / Low)
- Click any row → Detail Panel with score breakdown visualization

Candidate Detail Panel

- Score breakdown (6 dimensions with weighted bars)

Performance & Evaluation

Runtime Benchmark

Component	Time
Candidate loading (100K)	1.0s
Feature extraction (75K scored)	49.4s
Ensemble scoring	1.3s
Reasoning generation	0.01s
CSV write + validate	0.02s
Total	51.7s

Validation Results

```
$ python validate_submission.py team_code liberators.csv
```

What Makes Us Different

Factor	Median Submission	Code Liberators
Dataset	Truncated (21K)	Full 100K
Title awareness	Keyword	Taxonomy (ideal→mismatch) + fuzzy
Skill scoring	Count-based	Quality: endorsements × duration
Honeypots	None or basic	8 checks, progressive penalty
Reasoning	Generic/templated	Score-derived, 20+ patterns
Behavioral	Ignored	Multiplicative calibration B(c)
Runtime	Unknown	52s proven on CPU
Tests	None	8-test E2E suite: 8/8 PASS
API	None	FastAPI, 9 endpoints, Swagger
Frontend	None	Full recruiter dashboard

Feature Ablation Study

What if we removed each feature module?

Removed Feature	Estimated NDCG@10 Impact
Title Match (30%)	–35% (largest loss — defines relevance)
Behavioral Multiplier	–15% (removes availability signal)
Semantic Score (10%)	–8% (reduces nuanced JD matching)
Career Trajectory (20%)	–20% (services-only penalty key)
Honeypot Penalty	–12% (honeypots enter top-100)
Skill Quality	–10% (reverts to keyword stuffing)

Key finding: Title weight is the dominant feature. The JD explicitly asks for AI specialists — our 30% title weight directly reflects this.

Future Improvements

Phase 2 (6 months)

- **Cross-encoder reranking:** `cross-encoder/ms-marco-MiniLM-L-12-v2` on top-500 candidates
- **BGE-M3 semantic search:** Upgrade from bge-small to bge-m3 for better recall
- **Dynamic JD parsing:** Extract weights from JD text using LLM rather than hard-coded

Phase 3 (12 months)

- **Learned ranking:** Train LambdaMART on historical recruiter feedback
- **Multi-role support:** Generalize to any JD, not just Senior AI Engineer
- **Real-time ranking:** Sub-100ms streaming update as new candidates join

Infrastructure

- **Kubernetes deployment:** Scale to 10M candidates

Repository Structure & Reproducibility

```
code_liberators/
├── rank.py                # Main entry point
├── rank_advanced.py       # Hybrid (semantic + FAISS)
├── precompute.py         # One-time embedding precomputation
├── team_code_liberators.csv # Final validated submission
├── submission_metadata.yaml # Real contact + metrics
├── config/
│   ├── title_tiers.yaml   # Title taxonomy (v2, bug-fixed)
│   ├── skill_ontology.yaml # 150+ skills with weights
│   └── behavioral_thresholds.yaml
├── src/
│   ├── features/          # 5 extraction modules
│   ├── scoring/           # Ensemble scorer
│   ├── explanation/       # Reasoning generator (grammar-fixed)
│   ├── embeddings/        # BGE model + FAISS index
│   └── api/main.py        # FastAPI: 9 endpoints
├── frontend/             # Full recruiter dashboard (React)
├── tests/                 # 8 E2E tests: 8/8 PASS
├── docs/
│   ├── FINAL_REPORT.md    # Requirements traceability matrix
│   └── deck.md            # This deck
```

Thank You

Team Code Liberators

Lead	Subhash Kumar
Email	subhash1403kumar@gmail.com
Phone	+91 6302181675
GitHub	github.com/SubhashKumar14/Redrob-AI-Ranker

Final Submission Summary

-  **100,000 candidates** ranked from the full official dataset
-  **52 seconds** on CPU, well under 5-minute limit